AORTHAR ACADEMY

Product, Engineering & QA Documentation

Comprehensive documentation pack for the three-subdomain education platform

April 3, 2026 v1.0.0 github.com/mos-ski/Aorthar-academy

Table of Contents

1. Product Overview

1.1 What This Product Is — PRD

1.2 User Types & Personas

1.3 System Architecture

1.4 Subdomain Routing & Route Groups

1.5 Core Features Summary

1.6 Database Tables (30+)

1.7 Third-Party Services

1.8 Classification

2. User Stories

2.1 Authentication & Onboarding (AS-001 to AS-006) — STORIES

2.2 University Student App (US-001 to US-010)

2.3 Standalone Courses (SC-001 to SC-005)

2.4 Admin Dashboard (AD-001 to AD-014)

2.5 Settings & Account (ST-001 to ST-004)

2.6 Notifications & Email (NT-001 to NT-003)

2.7 Partnership (PT-001)

3. Module PRDs

3.1 Authentication & Onboarding — PRD

3.2 University Student App

3.3 Standalone Courses

3.4 Admin Dashboard

3.5 Payments & Monetization

3.6 Settings & Account

3.7 Notifications & Email

3.8 Internship (Planned)

4. QA Test Cases

4.1 Smoke Test Suite (10 tests) QA

4.2 Authentication (14 tests)

4.3 University Student App (19 tests)

4.4 Standalone Courses (14 tests)

4.5 Admin Dashboard (22 tests)

4.6 Payments (9 tests)

4.7 Security (7 tests)

4.8 Edge Cases & Error Handling (8 tests)

4.9 Regression Scenarios (7 scenarios)

5. System Architecture

5.1 High-Level Architecture ARCH

5.2 Data Flow Diagrams

5.3 Database Schema Overview

5.4 Deployment Architecture

5.5 Security Model

5.6 Performance Considerations

6. Gap Analysis

6.1 Critical Gaps ANALYSIS

6.2 Important Gaps

6.3 Nice-to-Have Gaps

6.4 Broken / Incomplete Flows

6.5 Mocked / Placeholder Logic

6.6 Backend Gaps

7. Technical Debt

7.1 Structural Issues ANALYSIS

7.2 Repeated Logic

7.3 Hardcoded Values

7.4 State Management Issues

7.5 Scalability Risks

7.6 Security Risks

7.7 Code Quality

7.8 Priority Fix Order

8. One-Year Roadmap

8.1 Q1 (Apr–Jun 2026) — Stabilization & Core Fixes ROADMAP

8.2 Q2 (Jul–Sep 2026) — Core Feature Expansion

8.3 Q3 (Oct–Dec 2026) — Growth & Monetization

8.4 Q4 (Jan–Mar 2027) — Scale & Optimization

8.5 Out of Scope

9. Product Recommendations

9.1 High-Impact, Low-Effort ANALYSIS

9.2 High-Impact, Medium-Effort

9.3 Medium-Impact, Medium-Effort

9.4 Strategic Recommendations

9.5 Quick Wins

10. Scope Control

10.1 In Scope SCOPE

10.2 Out of Scope

10.3 Grey Area

1. Full Product Overview

Project Summary

Stack: Next.js 16 App Router · TypeScript · Tailwind CSS v4 · shadcn/ui · Supabase · Paystack · Gemini AI · Resend

Deployment: Vercel (aorthar.com, courses.aorthar.com, university.aorthar.com)

Classification: Full-stack production product (not a prototype)

Last Updated: April 3, 2026

1.1 What This Product Is

Aorthar Academy is a **three-subdomain education product** built on a single Next.js codebase, sharing Supabase Auth via `.aorthar.com` cookies:

Platform 1: Academy (aorthar.com)

The main platform — marketing landing page, pricing, authentication, student dashboard, classroom, admin CMS, and the structured 4-year academic program. Students enroll in a department, progress through year/semester courses, take quizzes/exams, earn grades, track GPA, and submit a capstone project. **Freemium model** — Years 100–300 are free, Year 400 + capstone require a premium subscription.

Platform 2: Standalone Courses (courses.aorthar.com)

A pay-per-course platform for self-paced learning. Individual courses with YouTube/Google Drive-powered lessons, lifetime access, one-time purchase via Paystack. Targets professionals who want specific skills without committing to the full university program.

Platform 3: University (university.aorthar.com)

The university-branded subdomain that serves the **same core dashboard/classroom experience** as aorthar.com, but with a distinct URL for branding purposes. It shares all routes, database tables, and authentication with the main academy — it is not a separate product but a branded entry point.

Platform 4: Internship (internship.aorthar.com) — PLANNED

A quarterly cohort program. Applicants buy a form (₦10k), take an online exam, top 10 get placed at startups for 3 months. **Database schema exists, only landing page is implemented.** Full application, exam, results, and placement flows are documented but not yet built.

1.2 User Types & Personas

PERSONA	DESCRIPTION
Visitor	Unauthenticated user browsing landing pages, pricing, course catalogs
Student (University)	Enrolled in a department, progressing through the 4-year curriculum. Free tier + optional premium upgrade
Student (Standalone)	Purchased one or more individual courses on courses.aorthar.com. No department enrollment required
Contributor	A student who has had 3+ suggestions approved. Auto-promoted. Can create curriculum proposals
Admin (Super Admin)	Full access to all admin features, user management, pricing, audit logs, ops hub
Admin (Content Admin)	Can manage courses, lessons, questions, curriculum, suggestions, capstone reviews
Admin (Finance Admin)	Can view payments, transactions, audit logs, pricing configuration
Suspended User	A user whose account has been suspended by an admin. Redirected to /suspended on login

1.3 System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Vercel (CDN + SSR)                            │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐  │
│  │  aorthar.com     │  │ courses.         │  │ university.      │  │
│  │  (Academy)       │  │ aorthar.com      │  │ aorthar.com      │  │
│  │  Next.js 16      │  │ (Standalone)     │  │ (University)     │  │
│  └──────────┬───────┘  └──────────┬───────┘  └──────────┬───────┘  │
│             └──────────────────────┼──────────────────────┘          │
│                                    ┌──────▼──────┐                   │
│                                    │  Next.js App │                   │
│                                    │  Router      │                   │
│                                    │  (shared)    │                   │
│                                    └──────┬──────┘                   │
└──────────────────────────────────────────┼──────────────────────────┘
                                           │
                                           │
                                           │
```

SUBDOMAIN	PURPOSE	AUTH	STATUS
`aorthar.com`	Main academy — landing, pricing, auth, dashboard, classroom, admin CMS	Public / Required	Live
`courses.aorthar.com`	Standalone pay-per-course platform (YouTube/Drive lessons)	Optional	Live
`university.aorthar.com`	University-branded entry point (same routes as aorthar.com)	Public / Required	Live
`internship.aorthar.com`	Quarterly cohort program (landing page only)	Public	Planned
`admin.aorthar.com`	Dedicated admin subdomain (currently /admin on main domain)	Admin Required	Planned

ROUTE GROUP	PATH	AUTH	PURPOSE
`(auth)`	/login, /register, /verify, /forgot-password, /reset-password	Public	Authentication pages
`(dashboard)`	/dashboard, /courses, /progress, /gpa, /capstone, /settings, /suggest	Required + onboarding gate	Student university app
`(classroom)`	/classroom/[courseId]	Required	Full-screen course viewer
`(admin)`	/admin/**	Required (admin guard, disabled in dev)	Admin CMS
`(courses-app)`	/courses-app/**	Optional (required for purchase)	Standalone courses platform
`(internship)`	/internship	Public	Internship landing page
root	/, /pricing, /about, /contact, /partnership, /terms, /privacy, /cookies	Public	Marketing & legal pages

1.5 Core Features Summary

FEATURE	STATUS	NOTES
Auth (email/password + verification)	Complete	Supabase Auth with email redirect
Student onboarding (department selection)	Complete	8 departments
4-year curriculum with progression gates	Complete	Years 100→400, semester unlocks
Quiz/exam engine with grading	Complete	Edge Function for server-side grading
GPA calculation (5.0 scale)	Complete	Semester + cumulative GPA
Capstone submission + review	Complete	Premium + 400-level + GPA ≥ 3.5
Community suggestions system	Complete	Vote, review, auto-promote contributors
Premium subscriptions (Paystack)	Complete	Semester/monthly/yearly/lifetime plans
Standalone courses (pay-per-course)	Complete	YouTube/Drive lessons, lifetime access
Admin CMS (courses, users, payments)	Complete	Multi-level admin permissions
Audit logging	Complete	Immutable audit trail
AI lesson content (Gemini)	Complete	Summaries, related content, deep-dives
Email (Resend)	Complete	Welcome, password reset, purchase confirmation
Demo mode	Complete	Cookie-based toggle, hidden in production
Account suspension	Complete	Admin can suspend/unsuspend
Transcript HTML download	Complete	Server-side HTML generation
Partnership inquiries	Complete	Form + database storage
Lesson interactions	Complete	Reactions, comments, comment reactions
Internship product	Planned	Database schema exists, landing page only

1.6 Database Tables (30+)

CATEGORY	TABLES
Core	profiles, years, semesters, courses, lessons, resources, questions
Assessment	quiz_attempts, course_grades, semester_gpas, cumulative_gpas
Progression	enrollments, user_progress, semester_progress, course_prerequisites
Capstone	capstone_submissions, specialization_tracks, course_tracks
Community	suggestions, suggestion_votes, lesson_reactions, lesson_comments, lesson_comment_reactions
Monetization	plans, subscriptions, transactions, standalone_courses, standalone_lessons, standalone_purchases, standalone_lesson_progress
Admin	audit_log, notifications, webhook_events, partnership_inquiries
AI Content	lesson_summaries, lesson_deep_dive_links
Internship (Planned)	internship_cohorts, internship_applications, internship_exam_results, internship_placements

1.7 Third-Party Services

SERVICE	PURPOSE	INTEGRATION STATUS
Supabase	Auth, Database, RLS, Edge Functions	Full integration
Paystack	Payment processing (NGN)	Full integration
Resend	Transactional emails	Full (3 templates)
Google Gemini 2.0 Flash	AI lesson summaries, quiz generation	Full integration
YouTube	Video embedding, transcripts, link healing	Full integration
Google Drive	Video embedding for standalone courses	Full integration

1.8 Classification

Full-Stack Production Product

This is **not a prototype**. It has a real Supabase database with 30+ tables and RLS policies, real Paystack payment processing with webhook handling, real Supabase Edge Functions for quiz grading/payment verification/GPA calculation/progression checking, real Resend email delivery, real Gemini AI integration, comprehensive admin CMS with role-based access, and a demo mode for development/testing with mock data snapshots.

The only incomplete product is the **Internship platform**, which has database tables and documentation but only a landing page implemented.

2. User Stories

> **Format**
>
> Each story follows: **As a [persona], I want [goal] so that [benefit].**
>
> Acceptance criteria use Gherkin format: **Given / When / Then**

2.1 Authentication & Onboarding

AS-001 — As a visitor, I want to browse the landing page and pricing so that I can understand what Aorthar Academy offers before signing up.

```
Given I am on aorthar.com
When I view the landing page
Then I see hero section, features, pricing cards, and partner logos
And I can navigate to login or register
```

AS-002 — As a visitor, I want to create an account with my email, name, and password so that I can access the platform.

```
Given I am on the register page
When I fill in my full name, email, and a valid password (8+ chars, uppercase, number)
And I click "Create Account"
Then I am redirected to the email verification page
And a verification email is sent to my inbox
```

AS-003 — As a new student, I want to verify my email so that I can activate my account.

```
Given I have received a verification email
When I click the verification link
Then my account is activated
And I am redirected to the onboarding page (university) or dashboard (courses)
```

AS-004 — As a university student, I want to select my department during onboarding so that I see the correct curriculum for my field.

```
Given I have verified my email on the university domain
When I complete the onboarding form with my department
Then my profile is updated with the department
And I am enrolled in the corresponding courses
And I am redirected to the dashboard
```

AS-005 — As a returning user, I want to log in with my email and password so that I can access my account.

```
Given I have a verified account
When I enter my email and password on the login page
And I click "Sign In"
Then I am redirected to my dashboard (or the ?next URL)
And my session persists across subdomains
```

AS-006 — As a user who forgot my password, I want to reset it via email so that I can regain access to my account.

```
Given I am on the forgot password page
When I enter my registered email
And I submit the form
Then I receive a password reset email
And clicking the link takes me to the reset password page
```

2.2 University Student App

US-001 — As a university student, I want to see my dashboard with my current semester, courses, and progress so that I know what to study next.

```
Given I am logged in and have completed onboarding
When I visit /dashboard
Then I see my current year/semester, enrolled courses, progress summary, and quick actions
```

US-002 — As a student, I want to view my course list so that I can see all available courses for my department and level.

```
Given I am on the courses page
When I view the course listing
Then I see courses grouped by semester with status indicators (not started, in progress, passed, failed)
```

US-003 — As a student, I want to enter the classroom for a course so that I can study lessons and take quizzes.

```
Given I am on a course page
When I click "Enter Classroom"
Then I am taken to the classroom viewer with lessons, materials, and quiz tabs
```

US-004 — As a student, I want to take a quiz so that I can test my knowledge and earn a grade.

```
Given I am in the classroom for a course
When I start a quiz
Then I see timed questions one at a time
And I can submit my answers
And I receive my score after submission
And correct answers are not exposed in the API response
```

US-005 — As a student, I want to take an exam so that I can complete the course assessment.

```
Given I have completed the quiz requirements for a course
When I start an exam
Then I see timed exam questions
And my score contributes to my final course grade
```

US-006 — As a student, I want to see my GPA so that I can track my academic performance.

```
Given I have completed at least one semester's courses
When I visit the GPA page
Then I see my semester GPA and cumulative GPA on a 5.0 scale
```

US-007 — As a student, I want to see my progress across semesters so that I know what I've completed and what's next.

```
Given I am on the progress page
Then I see a visual breakdown of my completed, in-progress, and locked courses by semester
```

US-008 — As a 400-level student with premium access and GPA ≥ 3.5, I want to submit my capstone project so that I can graduate.

```
Given I meet all capstone requirements (400-level, premium, GPA ≥ 3.5)
When I fill out the capstone form with GitHub URL, live URL, description, and tech stack
And I submit it
Then my capstone is marked as pending review
```

US-009 — As a student, I want to suggest improvements to the curriculum so that I can contribute to the community.

```
Given I am logged in
When I submit a suggestion for a course, lesson, or resource
Then my suggestion is recorded as pending review
And I earn contributor status after 3 approved suggestions
```

US-010 — As a student, I want to upgrade to premium so that I can access Year 400 courses and the capstone.

```
Given I am on the pricing page
When I select a premium plan and complete payment via Paystack
Then my subscription is activated
And I gain access to premium features
```

2.3 Standalone Courses (courses.aorthar.com)

SC-001 — As a visitor, I want to browse standalone courses so that I can find courses to purchase.

```
Given I visit courses.aorthar.com
When I view the course catalog
Then I see published courses with titles, descriptions, prices, and instructor info
```

SC-002 — As a visitor, I want to preview the first lesson of a course for free so that I can evaluate the course quality.

```
Given I am viewing a course page
When I play the first lesson
Then I can watch a 1-minute free preview
And after the preview, I see a paywall prompt
```

SC-003 — As a logged-in user, I want to purchase a course so that I get lifetime access.

```
Given I am logged in and viewing a course
When I click "Buy this course"
Then I am redirected to Paystack to complete payment
And after successful payment, I gain full access to all lessons
And a purchase confirmation email is sent
```

SC-004 — As a course buyer, I want to watch purchased lessons so that I can learn at my own pace.

```
Given I have purchased a course
When I visit the course learn page
Then I can watch all lessons without restrictions
And my progress is tracked
```

SC-005 — As a learner, I want to see my purchased courses in one place so that I can easily resume learning.

```
Given I have purchased one or more courses
When I visit /courses-app/learn
Then I see a list of all my purchased courses with progress indicators
```

2.4 Admin Dashboard

AD-001 — As an admin, I want to see an overview dashboard so that I can quickly understand platform health.

```
Given I am logged in as an admin
When I visit /admin
Then I see key metrics: total users, active subscriptions, revenue, recent activity
```

AD-002 — As an admin, I want to manage courses so that I can create, edit, and publish curriculum content.

```
Given I am on the admin courses page
When I create or edit a course
Then I can set the code, name, description, thumbnail, year, semester, credit units, pass mark, and grading
weights
```

AD-003 — As an admin, I want to manage lessons and resources within a course so that I can structure the learning content.

```
Given I am editing a course
When I add lessons and resources
Then I can set titles, content (Markdown), YouTube URLs, links, and documents
And I can order them
```

AD-004 — As an admin, I want to manage the question bank so that I can create quiz and exam questions.

```
Given I am on the admin questions page
When I create a question
Then I can set the question text, options (with correct answers), explanation, points, difficulty, and scope
```

AD-005 — As an admin, I want to view and manage users so that I can see who has registered, their roles, and their purchases.

```
Given I am on the admin users page
When I view the user list
Then I see all registered users with their name, email, role, department, premium status, and course purchases
And I can search and filter by role
```

AD-006 — As an admin, I want to view all transactions so that I can monitor revenue and payment history.

```
Given I am on the admin payments page
When I view the transaction list
Then I see both university subscriptions and standalone course purchases with user info, amounts, and dates
```

AD-007 — As an admin, I want to review student suggestions so that I can approve or reject curriculum improvements.

```
Given I am on the admin suggestions page
When I review a suggestion
Then I can see the proposed content, vote count, and approve or reject with notes
```

AD-008 — As an admin, I want to review capstone submissions so that I can approve or request revisions.

```
Given I am on the admin capstone page
When I review a capstone
Then I can see the GitHub URL, live URL, description, tech stack, and approve/reject/request revision
```

AD-009 — As an admin, I want to manage standalone courses so that I can create and edit pay-per-course offerings.

```
Given I am on the admin standalone courses page
When I create a standalone course
Then I can set the slug, title, description, price, thumbnail, and status
And I can add lessons with YouTube/Drive URLs
```

AD-010 — As a super admin, I want to manage pricing plans so that I can adjust subscription pricing.

```
Given I am on the admin pricing page
When I edit a plan
Then I can change the name, price, billing type, and access scope
```

AD-011 — As a super admin, I want to view audit logs so that I can track all admin actions on the platform.

```
Given I am on the admin audit logs page
When I view the logs
Then I see a chronological list of all admin actions with who performed them, what changed, and when
```

AD-012 — As a super admin, I want to suspend a user's account so that I can block access for policy violations.

```
Given I am viewing a user in the admin panel
When I suspend their account
Then they are immediately redirected to the suspended page on next login
And their profile is marked as suspended
```

AD-013 — As a super admin, I want to grant premium access to a user so that I can give them free premium status.

```
Given I am viewing a user in the admin panel
When I grant them premium access
Then they immediately gain access to premium features
```

AD-014 — As a super admin, I want to grant standalone course access to a user so that I can give them a course without payment.

```
Given I am viewing a user in the admin panel
When I grant them access to a standalone course
Then they can immediately access that course's lessons
```

2.5 Settings & Account

ST-001 — As a user, I want to update my profile information so that my name and social links are current.

```
Given I am on the settings page
When I edit my full name, bio, GitHub URL, or LinkedIn URL
And I save
Then my profile is updated
```

ST-002 — As a student, I want to change my department after onboarding so that I can switch to a different curriculum.

```
Given I have completed onboarding
When I request a department change
Then my profile is updated with the new department
And my progress is recalculated for the new curriculum
```

ST-003 — As a user, I want to delete my account so that I can remove all my data from the platform.

```
Given I am on the settings page
When I confirm account deletion
Then my profile, progress, and submissions are deleted
And I am logged out
```

ST-004 — As a user, I want to download my academic transcript so that I have a record of my performance.

```
Given I have completed at least one course
When I request my transcript
Then an HTML file is generated and downloaded
And it includes my courses, grades, and GPA
```

2.6 Notifications & Email

NT-001 — As a student, I want to receive a welcome email after signing up so that I feel welcomed and know next steps.

```
Given I have just created an account
When my account is created
Then I receive a welcome email with next steps
```

NT-002 — As a course buyer, I want to receive a purchase confirmation email so that I have a record of my purchase.

```
Given I have successfully purchased a course
When the payment is verified
Then I receive a confirmation email with course details and access link
```

NT-003 — As a student, I want to receive a password reset email so that I can recover my account.

```
Given I requested a password reset
When I submit my email on the forgot password page
Then I receive an email with a reset link
```

2.7 Partnership

PT-001 — As a potential partner, I want to submit a partnership inquiry so that I can explore collaboration opportunities.

```
Given I am on the partnership page or contact form
When I fill out the partnership inquiry form with my name, company, email, type, and message
And I submit it
Then my inquiry is recorded in the database
And I see a success confirmation
```

3. Module PRDs

3.1 Authentication & Onboarding

Overview

Email/password authentication with email verification, password reset, and department-based onboarding for university students. Standalone course users skip department selection. All subdomains share Supabase Auth via `.aorthar.com` cookie domain.

Goals & Objectives

- Frictionless signup with email verification
- Automatic profile creation via database trigger
- Department routing for university students
- Cross-subdomain session sharing
- Suspended user blocking at login

User Personas

- **University Student:** Needs department selection + onboarding
- **Standalone Student:** Needs minimal signup (name + email + password)
- **Admin:** Needs secure access with role-based routing
- **Suspended User:** Must be blocked from all authenticated areas

User Stories

AS-001 through AS-006 (see Section 2)

User Flows

```
University Signup:
Visit aorthar.com/register → Fill name, email, password, department →
Verify email → Onboarding (department confirmation + semester setup) → Dashboard

Standalone Signup:
Visit courses.aorthar.com → Click "Sign up & enroll" → Fill name, email, password →
Verify email → Redirect to checkout → Paystack → Course access

Login:
Visit /login → Enter email + password → Check suspension status →
Redirect to ?next or default dashboard

Forgot Password:
Visit /forgot-password → Enter email → Receive reset email →
Click link → Enter new password → Redirect to dashboard
```

Functional Requirements

ID	REQUIREMENT
FR-001	Sign up with email, password (8+ chars, uppercase, number), full name
FR-002	Email verification required before access
FR-003	Password reset via email link
FR-004	Auto-create profile row via handle_new_user() trigger
FR-005	Department selection on university domain (8 options)
FR-006	Minimal signup on courses domain (no department)
FR-007	Session persistence across subdomains via .aorthar.com cookie
FR-008	Suspended users redirected to /suspended

Non-Functional Requirements

- Signup must complete in < 3 seconds
- Password reset email delivered within 30 seconds
- Session cookie secure, httpOnly, sameSite=lax
- Zod validation on all forms with inline error messages

Edge Cases / Unhappy Paths

- User tries to register with existing email → Supabase error shown
- User clicks expired verification link → Resend option shown
- User logs in from different device → Session created independently
- User is suspended → Redirected to /suspended with no bypass
- Network error during signup → Error message shown, form state preserved

Acceptance Criteria

- AC-001: User can register with valid credentials and receives verification email
- AC-002: User can verify email and access dashboard
- AC-003: User can login with correct credentials
- AC-004: User can reset password via email link
- AC-005: Suspended user is blocked from all authenticated areas

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`POST /api/auth/callback`	Real	Supabase email verification callback
`POST /api/auth/forgot-password`	Real	Send reset email via Resend
`POST /api/auth/send-welcome`	Real	Send welcome email via Resend
Supabase Auth	Real	signInWithPassword, signUp, updateUser

UI/UX Notes

- Split-panel layout on desktop (brand left, form right)
- Mobile: single column form
- Domain detection changes form fields (department shown on aorthar.com, hidden on courses.aorthar.com)
- Zod validation with inline error messages

Metrics / Success Criteria

- Signup completion rate
- Email verification rate
- Time to first login
- Password reset request rate

3.2 University Student App

Overview

The core university experience: 4-year curriculum, quiz/exam engine, GPA tracking, progression gates, and capstone submission. Served from both aorthar.com and university.aorthar.com (same routes, same database).

Goals & Objectives

- Structured learning path through department-specific curriculum
- Fair assessment with server-side grading and cooldowns
- Accurate GPA tracking on 5.0 scale
- Clear progression gates (semester/year unlocks)

User Personas

- **Year 100–300 Student:** Free access, progressing through curriculum
- **Year 400 Student:** Requires premium subscription + GPA ≥ 3.5
- **Contributor:** Auto-promoted after 3 approved suggestions

User Stories

US-001 through US-010 (see Section 2)

User Flows

```
Course Progression:
Dashboard → Select course → Enter classroom → Study lessons →
Start quiz → Answer questions → Submit → Get score →
Complete quiz requirements → Take exam → Get final grade →
Semester unlocks when all courses pass

GPA Calculation:
Complete courses → Grades computed (quiz 40% + exam 60%) →
Semester GPA calculated → Cumulative GPA updated →
Visible on /gpa page

Capstone:
400-level + premium + GPA ≥ 3.5 → Fill capstone form →
Submit → Status: pending → Admin reviews → Approved/Rejected/Revision
```

Functional Requirements

ID	REQUIREMENT
FR-001	Display courses by semester with status indicators
FR-002	Classroom with lessons, materials, related content, quiz tabs
FR-003	Timed quiz/exam with shuffled questions, no answer leakage
FR-004	Server-side grading via Supabase Edge Function
FR-005	Cooldown between attempts (default 24 hours)
FR-006	Attempt limits (default 3 per quiz/exam)
FR-007	Grade computation: quiz_weight × quiz_score + exam_weight × exam_score
FR-008	GPA on 5.0 scale with 8 grade points
FR-009	Semester unlock: all previous semester courses must pass
FR-010	Year 400 requires premium subscription
FR-011	Capstone requires 400-level + GPA ≥ 3.5 + premium

Non-Functional Requirements

- Quiz questions must never expose correct answers to client
- Grading must be server-side (Edge Function)
- Grade computation must be idempotent
- Classroom must load in < 2 seconds

Edge Cases / Unhappy Paths

- Student tries to take quiz during cooldown → Blocked with timer
- Student exceeds attempt limit → Blocked permanently for that assessment
- Student upgrades to premium mid-semester → Immediate access to Year 400
- Student changes department → Progress recalculated for new curriculum
- Quiz timer expires → Auto-submit or block further answers

Acceptance Criteria

- AC-001: Dashboard shows current semester courses and progress
- AC-002: Classroom loads with lessons, materials, and quiz tabs
- AC-003: Quiz starts with timer and shuffled questions
- AC-004: Quiz submission returns score without exposing correct answers

- AC-005: GPA page shows correct semester and cumulative GPA
- AC-006: Capstone submission requires 400-level + premium + GPA ≥ 3.5

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`POST /api/quiz/start`	Real	Create attempt, return shuffled questions
`POST /api/quiz/submit`	Real	Delegate to grade-quiz Edge Function
`GET /api/quiz/attempt/[id]`	Real	Fetch attempt state
`GET /api/quiz/attempt/[id]/solutions`	Real	Fetch graded solutions
`POST /api/quiz/generate`	Real	AI-generate questions (Gemini)
`GET /api/lessons/summary`	Real	AI lesson summary
`GET /api/lessons/related`	Real	AI related content
`GET /api/lessons/deep-dive`	Real	AI deep-dive links
`POST /api/lessons/reaction`	Real	Like/dislike lesson
`GET/POST /api/lessons/comments`	Real	Lesson comments
`POST /api/capstone/submit`	Real	Submit capstone
`POST /api/unlock-next-level`	Real	Trigger progression check

UI/UX Notes

- Dashboard: `px-[15%]` horizontal padding from layout
- Classroom: full-screen dark theme, no sidebar
- QuizRunner: timer in header, question navigation, submit confirmation
- GPA page: semester breakdown + cumulative summary

Metrics / Success Criteria

- Course completion rate
- Average quiz score
- Average exam score
- GPA distribution
- Capstone submission rate
- Premium conversion rate

3.3 Standalone Courses

Overview

Pay-per-course platform on courses.aorthar.com. One-time purchase via Paystack, lifetime access, YouTube/Drive-powered lessons.

Goals & Objectives

- Simple purchase flow with minimal friction
- Free preview to drive conversions
- Reliable purchase recording (webhook + redirect fallback)

- Progress tracking across lessons

User Personas

- **Prospective Buyer:** Browsing, previewing, deciding to purchase
- **Course Owner:** Has purchased, watching lessons
- **Multi-course Learner:** Has purchased multiple courses

User Stories

SC-001 through SC-005 (see Section 2)

User Flows

```
Purchase Flow:
Browse courses → Select course → Watch 1-min preview →
Click "Buy" → Paystack checkout → Payment success →
Redirect to /courses-app/learn/[slug] → Record purchase →
Access all lessons

Learning Flow:
Visit /courses-app/learn → See purchased courses →
Select course → Watch lessons in order → Progress tracked
```

Functional Requirements

ID	REQUIREMENT
FR-001	Display published standalone courses with pricing
FR-002	Free 1-minute preview of first lesson
FR-003	Paystack checkout with course metadata
FR-004	Idempotent purchase recording (upsert on paystack_reference)
FR-005	Purchase verification on redirect AND via webhook
FR-006	YouTube and Google Drive video embedding
FR-007	Lesson progress tracking
FR-008	"My Courses" page showing all purchased courses

Non-Functional Requirements

- Purchase must be recorded even if user closes browser (webhook)
- Video player must handle YouTube and Drive sources
- Preview timer must be client-side enforced (server validates access)

Edge Cases / Unhappy Paths

- User pays but doesn't return to site → Webhook records purchase
- User tries to access unpurchased lesson → Redirected to course page
- Duplicate payment (same reference) → Upsert prevents duplicate
- Course status changes to draft after purchase → Buyer retains access

Acceptance Criteria

- AC-001: Course catalog shows published courses with prices
- AC-002: Free preview plays for 1 minute then shows paywall
- AC-003: Purchase via Paystack grants lifetime access

- AC-004: Webhook records purchase even if user closes browser
- AC-005: My Courses page shows all purchased courses with progress

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`POST /api/standalone/checkout`	Real	Create Paystack session
`GET /api/standalone/verify-payment`	Real	Verify payment on redirect
`POST /api/standalone/progress`	Real	Track lesson progress
`POST /api/standalone/logout`	Real	Logout from standalone
`POST /api/webhooks/paystack`	Real	Webhook handler (records purchases)

UI/UX Notes

- Dark theme (#0f1011 background)
- Green accent (#a7d252)
- Sticky video player on desktop
- Mobile: stacked layout with video on top
- Auto-start checkout on first visit (sessionStorage flag)

Metrics / Success Criteria

- Course page views → purchase conversion rate
- Average revenue per buyer
- Preview completion rate
- Lesson completion rate
- Refund/chargeback rate

3.4 Admin Dashboard

Overview

Multi-level admin CMS for managing courses, users, payments, pricing, curriculum, and platform operations. Currently served from `/admin/**` on the main domain. Dedicated admin.aorthar.com subdomain is planned.

Goals & Objectives

- Complete content management for university and standalone courses
- User management with suspension and premium controls
- Financial visibility (transactions, revenue, conversion rates)
- Audit trail for all admin actions
- Role-based access control (super, content, finance)

User Personas

- **Super Admin:** Full access, user management, pricing, ops
- **Content Admin:** Course/lesson/question management, curriculum
- **Finance Admin:** Payment monitoring, audit logs, pricing view

User Stories

AD-001 through AD-014 (see Section 2)

User Flows

```
Course Creation:
Admin → /admin/courses → Create course → Set code, name, year, semester →
Add lessons → Add resources (YouTube/link/document) → Add questions →
Publish course

User Management:
Admin → /admin/users → Search/filter users → View user details →
Suspend/unsuspend → Grant premium → Grant standalone access

Payment Monitoring:
Admin → /admin/payments → View unified transaction table →
See subscriptions + standalone purchases → Filter by type →
Monitor revenue and conversion rates
```

Functional Requirements

ID	REQUIREMENT
FR-001	CRUD for courses, lessons, resources, questions
FR-002	User listing with search, role filter, premium/purchase status
FR-003	Transaction listing (subscriptions + standalone purchases)
FR-004	Suggestion review with approve/reject
FR-005	Capstone review with approve/reject/request revision
FR-006	Standalone course management (create, edit, publish, add lessons)
FR-007	Pricing plan configuration
FR-008	Audit log viewer (immutable)
FR-009	User suspension/unsuspension
FR-010	Premium grant/revoke
FR-011	Standalone course access grant
FR-012	Student invite and bulk import
FR-013	Admin access management (invite, grant, update level)

Non-Functional Requirements

- Admin routes protected by role check (disabled in development)
- All admin actions logged to audit_log table
- Audit log is append-only (no deletes/updates)

Edge Cases / Unhappy Paths

- Non-admin tries to access /admin → Redirected to /unauthorized
- Admin deletes a course with existing enrollments → RESTRICT or cascade
- Two admins edit same course simultaneously → Last write wins (no locking)
- Finance admin tries to edit courses → Blocked by permissions

Acceptance Criteria

- AC-001: Admin dashboard shows key metrics (users, subscriptions, revenue)
- AC-002: Admin can create/edit/delete courses, lessons, resources, questions

- AC-003: Admin can view/search/filter users with purchase status
- AC-004: Admin can view all transactions (subscriptions + standalone)
- AC-005: Admin can review suggestions and capstones
- AC-006: Non-admin users are blocked from /admin routes

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`GET/POST /api/admin/courses`	Real	List/create courses
`GET/PATCH/DELETE /api/admin/courses/[id]`	Real	Course CRUD
`GET/POST /api/admin/courses/[id]/lessons`	Real	Course lessons
`GET/POST /api/admin/courses/[id]/questions`	Real	Course questions
`PATCH/DELETE /api/admin/lessons/[id]`	Real	Lesson CRUD
`PATCH/DELETE /api/admin/questions/[id]`	Real	Question CRUD
`GET/PATCH /api/admin/suggestions/[id]`	Real	Review suggestion
`GET/PATCH /api/admin/capstone/[id]`	Real	Review capstone
`GET /api/admin/users`	Real	List users
`PATCH /api/admin/users/[id]`	Real	Update user role
`PATCH /api/admin/users/[id]/premium`	Real	Grant/revoke premium
`PATCH /api/admin/users/[id]/suspension`	Real	Suspend/unsuspend user
`GET /api/admin/payments`	Real	List transactions
`GET /api/admin/transactions/unified`	Real	Unified transaction view
`POST /api/admin/admin-access/invite`	Real	Invite admin
`PATCH /api/admin/admin-access/[id]/level`	Real	Update admin level
`GET /api/admin/audit-logs`	Real	View audit logs
`GET/PATCH /api/admin/pricing`	Real	Plan management
`GET/POST /api/admin/standalone-courses`	Real	Bootcamp CRUD

UI/UX Notes

- Admin guard disabled in development (NEXT_PUBLIC_APP_ENV=development)
- Sidebar navigation with module grouping
- Badge indicators for counts (pending suggestions, etc.)
- Environment badge (DEV/STAGING/PROD) in navbar

Metrics / Success Criteria

- Admin action frequency
- Time to review suggestions
- Time to review capstones
- User suspension rate

3.5 Payments & Monetization

Overview

Paystack-powered payment processing for university subscriptions and standalone course purchases. Webhook-based verification with idempotent recording. All transactions in NGN.

Goals & Objectives

- Reliable payment processing with webhook fallback
- Idempotent transaction recording
- Clear financial reporting for admins
- Multiple plan types (semester, monthly, yearly, lifetime)

User Personas

- **Student (University):** Purchasing premium subscription
- **Student (Standalone):** Purchasing individual courses
- **Finance Admin:** Monitoring revenue and transactions

User Stories

US-010, SC-003, AD-006 (see Section 2)

User Flows

```
University Subscription:
Student → /pricing → Select plan → /pricing/checkout/[planId] →
Paystack checkout → Payment success → Webhook fires →
Edge Function verifies → Subscription created → Access granted

Standalone Course:
Student → /courses-app/[slug] → Click "Buy" →
/api/standalone/checkout → Paystack → Payment success →
Redirect to /courses-app/learn/[slug] → Verify payment →
Record purchase → Access all lessons
```

Functional Requirements

ID	REQUIREMENT
FR-001	Paystack checkout session creation
FR-002	Webhook signature verification (HMAC-SHA512)
FR-003	Idempotent upsert on paystack_reference
FR-004	Subscription status tracking (active/expired/cancelled)
FR-005	Purchase confirmation emails
FR-006	Revenue calculation and display
FR-007	Conversion rate calculation

Non-Functional Requirements

- Webhook must handle duplicate events gracefully
- Payment verification must complete in < 5 seconds
- Transaction records are immutable

Edge Cases / Unhappy Paths

- Webhook fires before redirect verification → Upsert handles duplicate
- User pays but webhook fails → Manual reconciliation via admin panel
- Paystack returns success but amount is wrong → Log and alert
- Subscription expires → User loses premium access immediately

Acceptance Criteria

- AC-001: Paystack checkout session is created with correct metadata
- AC-002: Webhook signature is verified before processing
- AC-003: Duplicate webhook events are handled idempotently
- AC-004: Subscription status is tracked correctly
- AC-005: Purchase confirmation email is sent on successful payment

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`POST /api/payments/checkout`	Real	Create Paystack session
`POST /api/webhooks/paystack`	Real	Webhook handler
`POST /api/standalone/checkout`	Real	Standalone checkout
`GET /api/standalone/verify-payment`	Real	Redirect verification
`GET /api/admin/transactions/unified`	Real	Unified transactions

UI/UX Notes

- Pricing cards with clear plan comparison
- Paystack hosted checkout (redirect)
- Success state with course access redirect
- Admin payments page with revenue cards and transaction table

Metrics / Success Criteria

- Total revenue (NGN)
- Subscription MRR
- Course purchase revenue
- Conversion rate (free → paid)
- Failed transaction rate

3.6 Settings & Account

Overview

User profile management, department changes, account deletion, and transcript downloads.

Goals & Objectives

- Self-service profile management
- Safe account deletion with data cleanup
- Professional transcript generation

User Stories

ST-001 through ST-004 (see Section 2)

Functional Requirements

ID	REQUIREMENT
FR-001	Edit full name, bio, GitHub URL, LinkedIn URL
FR-002	Change department (recalculates progress)
FR-003	Delete account (cascading delete)
FR-004	Download transcript HTML

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`GET/PATCH /api/account`	Real	Read profile, delete account
`PATCH /api/profile`	Real	Update profile (name, bio)
`POST /api/onboarding/change-department`	Real	Change department
`GET /api/transcript/download`	Real	Generate transcript HTML

Edge Cases / Unhappy Paths

- User deletes account with active subscription → Subscription cancelled
- User changes department mid-semester → Progress recalculated
- User tries to delete account with pending capstone → Allowed (capstone deleted too)

Acceptance Criteria

- AC-001: User can update profile information
- AC-002: User can change department and progress is recalculated
- AC-003: User can delete account and all data is removed
- AC-004: User can download transcript with courses, grades, and GPA

3.7 Notifications & Email

Overview

Email delivery via Resend for welcome, password reset, and purchase confirmation. In-app notifications table exists but is not yet surfaced in the UI.

Status: Partial

Current Implementation

- Welcome email sent on registration (via `/api/auth/send-welcome`)
- Password reset email sent via Supabase Auth + Resend template
- Purchase confirmation email sent on successful payment (fire-and-forget)
- `notifications` table exists in schema but has no UI or API routes

Missing

- In-app notification bell in Navbar
- Notification list page
- Email notification when suggestion is approved/rejected

- Email notification when capstone is reviewed
- Push notifications or SMS

API / Data Requirements

ENDPOINT	STATUS	PURPOSE
`POST /api/auth/send-welcome`	Real	Welcome email
`POST /api/auth/forgot-password`	Real	Password reset email
Purchase confirmation (inline)	Real	Purchase email
In-app notifications UI	Missing	Notification bell + list page

---

3.8 Internship (Planned)

Overview

A quarterly cohort program on internship.aorthar.com. Applicants buy a form (₦10k), take an online exam, top 10 get placed at startups for 3 months. **Database schema exists, only landing page is implemented.**

Status: Planned — Not Built

What Exists

- Landing page at `/internship`
- Database tables: `internship_cohorts`, `internship_applications`, `internship_exam_results`, `internship_placements`
- Documentation: overview, architecture, user stories, module specs

What's Missing

- Application form and purchase flow
- Exam entry (name + code)
- Active exam session
- Results lookup
- Placement management
- All API routes (`/api/internship/*`)

Planned User Stories

- As an applicant, I want to purchase the exam form so that I can apply for the internship
- As an applicant, I want to take the online exam so that I can be evaluated
- As an applicant, I want to check my results so that I know if I was selected
- As an admin, I want to manage cohorts and placements so that I can run the program

4. QA Test Cases

Test Suite Overview

Total Test Cases: 100+ across 8 modules

Smoke Tests: 10 critical path tests

Regression Scenarios: 7 scenarios

Priority Distribution: High (40%), Medium (45%), Low (15%)

4.1 Smoke Test Suite (Critical Path)

#	TEST	MODULE	PRIORITY
SM-01	User can register with valid credentials	Auth	High
SM-02	User receives verification email	Auth	High
SM-03	User can verify email and access dashboard	Auth	High
SM-04	User can log in with valid credentials	Auth	High
SM-05	Dashboard loads with courses	University	High
SM-06	User can start and submit a quiz	University	High
SM-07	User can purchase a standalone course	Standalone	High
SM-08	Admin can view users and transactions	Admin	High
SM-09	Payment webhook records purchase	Payments	High
SM-10	Suspended user cannot access platform	Auth	High

4.2 Authentication (14 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
AUTH-01	Register with valid university credentials	High	1. Visit /register on aorthar.com 2. Enter name, email, valid password, department 3. Submit	Account created, redirected to /verify, email sent
AUTH-02	Register with valid standalone credentials	High	1. Visit /register on courses.aorthar.com 2. Enter name, email, valid password 3. Submit	Account created, redirected to /verify, email sent
AUTH-03	Register with weak password	Medium	1. Enter password "password1" (no uppercase) 2. Submit	Error: "Must contain at least one uppercase letter"
AUTH-04	Register with existing email	Medium	1. Use email of existing account 2. Submit	Supabase error shown to user
AUTH-05	Register with invalid email	Low	1. Enter "notanemail" 2. Submit	Error: "Invalid email address"
AUTH-06	Login with correct credentials	High	1. Visit /login 2. Enter valid email + password 3. Submit	Redirected to dashboard
AUTH-07	Login with wrong password	Medium	1. Enter valid email + wrong password 2. Submit	Error shown, no redirect
AUTH-08	Login with unverified email	Medium	1. Register new account 2. Don't verify 3. Try to login	Login succeeds but onboarding gate blocks access
AUTH-09	Forgot password flow	High	1. Visit /forgot-password 2. Enter email 3. Submit	Reset email sent, success message shown
AUTH-10	Reset password	High	1. Click reset link from email 2. Enter new password (matching confirm) 3. Submit	Password updated, redirected to dashboard
AUTH-11	Reset password with mismatched passwords	Low	1. Enter different passwords in both fields 2. Submit	Error: "Passwords do not match"
AUTH-12	Suspended user login	High	1. Suspend user via admin 2. User logs in	Redirected to /suspended
AUTH-13	Session persistence across subdomains	High	1. Login on aorthar.com 2. Visit courses.aorthar.com	User is authenticated on both
AUTH-14	Logout from standalone platform	Medium	1. On courses.aorthar.com 2. Call /api/standalone/logout	Session cleared, redirected

4.3 University Student App (19 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
UNI-01	Dashboard loads with semester courses	High	1. Login as student 2. Visit /dashboard	Shows current year/semester, courses, progress
UNI-02	Course listing shows all semesters	Medium	1. Visit /courses	Courses grouped by semester with status
UNI-03	Enter classroom for a course	High	1. Click course → Enter Classroom	Classroom loads with lessons, materials tabs
UNI-04	Start quiz	High	1. In classroom, click "Start Quiz" 2. Confirm	Quiz starts, timer begins, questions shown
UNI-05	Submit quiz	High	1. Answer questions 2. Click Submit	Score calculated, results displayed
UNI-06	Quiz answers not exposed to client	High	1. Start quiz 2. Inspect network requests	No correct answers in API response
UNI-07	Quiz cooldown enforcement	Medium	1. Submit quiz 2. Try to start again immediately	Blocked with cooldown timer
UNI-08	Quiz attempt limit enforcement	Medium	1. Submit quiz 3 times 2. Try 4th attempt	Blocked, no new attempt created
UNI-09	GPA page shows correct calculation	High	1. Complete courses with known scores 2. Visit /gpa	GPA matches manual calculation
UNI-10	Progress page shows semester status	Medium	1. Visit /progress	Visual breakdown of completed/locked courses
UNI-11	Capstone submission (eligible student)	High	1. 400-level + premium + GPA ≥ 3.5 2. Fill capstone form 3. Submit	Status changes to "pending"
UNI-12	Capstone submission (ineligible student)	Medium	1. Not 400-level or no premium 2. Try to submit	Form disabled or blocked
UNI-13	Submit curriculum suggestion	Medium	1. Visit /suggest 2. Fill form 3. Submit	Suggestion recorded as pending
UNI-14	Premium subscription purchase	High	1. Visit /pricing 2. Select plan 3. Complete Paystack	Subscription activated, premium features unlocked
UNI-15	Semester unlock after passing all courses	High	1. Pass all Semester 1 courses 2. Refresh	Semester 2 courses unlocked
UNI-16	Year 400 locked without premium	Medium	1. Free student 2. Try to access 400-level	Blocked, prompted to upgrade
UNI-17	Department change	Medium	1. Settings → Change department 2. Select new 3. Confirm	Profile updated, progress recalculated
UNI-18	Download transcript	Medium	1. Complete at least one course 2. Request transcript	HTML file downloaded with grades and GPA
UNI-19	Delete account	High	1. Settings → Delete account 2. Confirm	Account deleted, logged out, data removed

4.4 Standalone Courses (14 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
SC-01	Browse published courses	High	1. Visit /courses-app	Published courses displayed with prices
SC-02	View course detail page	High	1. Click a course	Course page with preview, description, buy button
SC-03	Free preview (1 minute)	High	1. Play first lesson without purchasing 2. Wait 60 seconds	Preview ends, paywall overlay shown
SC-04	Purchase course via Paystack	High	1. Logged in 2. Click "Buy" 3. Complete Paystack	Redirected to learn page, purchase recorded
SC-05	Purchase recording (webhook)	High	1. Purchase course 2. Close browser immediately 3. Wait for webhook	Purchase recorded in database
SC-06	Access purchased lessons	High	1. Purchase course 2. Visit /courses-app/learn/[slug]	All lessons accessible, no paywall
SC-07	Access unpurchased course	Medium	1. Don't purchase 2. Try /courses-app/learn/[slug]	Redirected to course page
SC-08	Lesson progress tracking	Medium	1. Watch lesson 2. Move to next 3. Check progress	Previous lesson marked complete
SC-09	My Courses page	Medium	1. Visit /courses-app/learn	List of all purchased courses with progress
SC-10	Duplicate purchase prevention	Medium	1. Purchase course 2. Try to purchase again	Redirected to learn page (409)
SC-11	YouTube video playback	Medium	1. Watch YouTube lesson	Video plays correctly
SC-12	Google Drive video playback	Medium	1. Watch Drive lesson	Video plays correctly
SC-13	Course not found (invalid slug)	Low	1. Visit /courses-app/nonexistent	404 page
SC-14	Draft course not visible	Low	1. Course status = draft 2. Browse courses	Course not shown

4.5 Admin Dashboard (22 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
AD-01	Admin dashboard loads	High	1. Login as admin 2. Visit /admin	Metrics displayed: users, subscriptions, revenue
AD-02	Create course	High	1. /admin/courses → Create 2. Fill form 3. Save	Course created, appears in list
AD-03	Edit course	Medium	1. Select course 2. Edit fields 3. Save	Changes saved and reflected
AD-04	Delete course	Medium	1. Select course 2. Delete	Course removed (or blocked if has enrollments)
AD-05	Add lesson to course	Medium	1. Edit course 2. Add lesson 3. Save	Lesson appears in course
AD-06	Add question to course	Medium	1. Edit course 2. Add question with options 3. Save	Question saved with correct options format
AD-07	View users list	High	1. /admin/users	All users shown with name, email, role, status
AD-08	Search users	Medium	1. /admin/users 2. Enter name/email in search	Filtered results
AD-09	Filter users by role	Medium	1. /admin/users 2. Select role filter	Only matching users shown
AD-10	View standalone course buyers	High	1. /admin/users 2. Check users with purchases	Buyers shown with course names in Purchases column
AD-11	View payments/transactions	High	1. /admin/payments	All subscriptions + standalone purchases shown
AD-12	Revenue calculation	Medium	1. /admin/payments 2. Check total revenue	Matches sum of all successful transactions
AD-13	Review suggestion	Medium	1. /admin/suggestions 2. Select pending 3. Approve/reject	Status updated, contributor promoted if applicable
AD-14	Review capstone	Medium	1. /admin/capstone 2. Select pending 3. Approve/reject/revision	Status updated, student notified
AD-15	Manage standalone courses	High	1. /admin/standalone-courses 2. Create course 3. Add lessons	Course created and published
AD-16	Suspend user	High	1. /admin/users 2. Suspend user	User redirected to /suspended on next login
AD-17	Grant premium access	Medium	1. /admin/users 2. Grant premium	User gains premium features immediately
AD-18	Grant standalone access	Medium	1. /admin/users 2. Grant course access	User can access course without payment
AD-19	View audit logs	Medium	1. /admin/audit-logs	Chronological log of admin actions
AD-20	Non-admin access blocked	High	1. Login as student 2. Visit /admin	Redirected to /unauthorized (in staging/prod)

ID	TITLE	PRIORITY	STEPS	EXPECTED
AD-21	Finance admin cannot edit courses	Medium	1. Login as finance_admin 2. Try to edit course	Blocked by permissions
AD-22	Content admin cannot view payments	Medium	1. Login as content_admin 2. Try /admin/payments	Blocked by permissions

4.6 Payments (9 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
PAY-01	Paystack checkout session created	High	1. Click checkout 2. API call	Payment URL returned
PAY-02	Webhook signature verification	High	1. Send webhook with invalid signature	400 response
PAY-03	Webhook records subscription	High	1. Successful subscription payment 2. Webhook fires	Subscription created in DB
PAY-04	Webhook records standalone purchase	High	1. Successful course payment 2. Webhook fires	Purchase recorded in DB
PAY-05	Idempotent webhook handling	High	1. Send same webhook event twice	Only one record created
PAY-06	Failed payment handling	Medium	1. Payment fails at Paystack	No subscription/purchase created
PAY-07	Purchase confirmation email sent	Medium	1. Complete purchase	Email received with course details
PAY-08	Subscription expiration	Medium	1. Subscription end_date passes 2. expire_subscriptions() runs	Status changed to expired
PAY-09	Premium access revoked on expiry	High	1. Subscription expires 2. Try to access premium feature	Blocked

4.7 Security (7 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
SEC-01	RLS prevents cross-user data access	High	1. Login as user A 2. Try to query user B's data	No data returned
SEC-02	Admin API requires admin role	High	1. Login as student 2. Call /api/admin/courses	401/403 response
SEC-03	Quiz answers not leaked	High	1. Start quiz 2. Inspect API response	No is_correct in options
SEC-04	Suspended user API blocked	High	1. Suspend user 2. Call any authenticated API	403 response
SEC-05	Paystack webhook HMAC verification	High	1. Send webhook without valid signature	400 response
SEC-06	Service role key not exposed	High	1. Inspect client bundle	No SUPABASE_SERVICE_ROLE_KEY
SEC-07	Demo mode hidden in production	Medium	1. Visit production site	No demo toggle visible

4.8 Edge Cases & Error Handling (8 tests)

ID	TITLE	PRIORITY	STEPS	EXPECTED
ERR-01	Network error during signup	Medium	1. Block network 2. Try to register	Error message shown
ERR-02	Paystack timeout during checkout	Medium	1. Start checkout 2. Network drops	Error message, retry option
ERR-03	Quiz timer expires	Medium	1. Start quiz 2. Wait until timer hits 0	Auto-submit or block further answers
ERR-04	Invalid course code format	Low	1. Admin creates course with "invalid-code"	Validation error
ERR-05	GPA with no courses	Low	1. New student visits /gpa	Shows 0.0 or "No grades yet"
ERR-06	Empty transaction list	Low	1. Admin visits /admin/payments with no transactions	"No transactions yet" message
ERR-07	Empty user list	Low	1. Admin visits /admin/users with no users	"No users" message
ERR-08	Concurrent quiz submissions	Low	1. Submit quiz twice rapidly	Only one attempt recorded

4.9 Regression Scenarios (7 scenarios)

#	SCENARIO	PRIORITY
REG-01	After deploying new course, existing students see it	High
REG-02	After changing pricing, existing subscriptions unaffected	High
REG-03	After department change, student's progress recalculated	High
REG-04	After admin suspension, user immediately blocked	High
REG-05	After webhook reprocessing, no duplicate transactions	High
REG-06	After updating quiz questions, existing attempts unaffected	Medium
REG-07	After adding new lesson, course progress not reset	Medium

Test Environment Requirements

- **Supabase project** with all migrations applied
- **Paystack test mode** with test keys
- **Resend test mode** or email sandbox
- **Gemini API key** for AI features
- **Test accounts:** 1 admin, 1 student (free), 1 student (premium), 1 standalone buyer
- **Browser:** Chrome, Safari, Firefox (latest)
- **Mobile:** iOS Safari, Android Chrome

5. System Architecture

5.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Vercel Edge Network                         │
│                                                                      │
│  ┌──────────────────┐              ┌──────────────────────────┐     │
│  │  aorthar.com     │              │  courses.aorthar.com     │     │
│  │  (University)    │              │  (Standalone Courses)    │     │
│  │                  │              │                          │     │
│  │  Next.js 16 SSR  │              │  Next.js 16 SSR          │     │
│  │  (same codebase) │              │  (same codebase)         │     │
│  └──────────────────┘              └──────────────────────────┘     │
│            │                                   │                     │
│            └───────────────────────────────────┘                     │
│                              │                                       │
│                    ┌─────────▼─────────┐                             │
│                    │  Next.js App      │                             │
│                    │  Router           │                             │
│                    │                   │                             │
│                    │  Route Groups:    │                             │
│                    │  - (auth)         │                             │
│                    │  - (dashboard)    │                             │
│                    │  - (classroom)    │                             │
│                    │  - (admin)        │                             │
│                    │  - (courses-app)  │                             │
│                    │  - university     │                             │
│                    │  - root           │                             │
│                    └───────────────────┘                             │
└──────────────────────────────────────────────────────────────────────┘
                                │
                                │
                ┌───────────────┼───────────────┐
                │               │               │
        ┌───────▼───────┐  ┌────▼────┐  ┌──────▼──────┐
        │  Supabase     │  │ Paystack │  │  Resend     │
        │               │  │         │  │             │
        │  • Auth       │  │ Payments │  │  Email      │
        │  • Postgres   │  │ Webhooks │  │  Templates  │
        │  • Edge Fns   │  │ NGN      │  │             │
        │  • Storage    │  │         │  │             │
        └───────────────┘  └─────────┘  └─────────────┘
                │
        ┌───────▼───────┐
        │  Gemini AI    │
        │  (generative) │
        │               │
        │  • Summaries  │
        │  • Related    │
        │  • Deep-dives │
        │  • Quiz gen   │
        └───────────────┘
```

5.2 Data Flow Diagrams

Authentication Flow

```
Client → Supabase Auth → Session cookie (.aorthar.com) →
Next.js validates → Server Component fetches profile →
Render page
```

Payment Flow (Standalone)

```
Client → POST /api/standalone/checkout → Paystack →
User pays → Two paths:
  1. Redirect → GET /api/standalone/verify-payment → Record purchase
  2. Webhook → POST /api/webhooks/paystack → Record purchase (idempotent)
→ Access granted
```

Payment Flow (University Subscription)

```
Client → POST /api/payments/checkout → Paystack →
User pays → Webhook → POST /api/webhooks/paystack →
Forward to Edge Function (verify-payment) →
Create subscription + transaction → Access granted
```

Quiz Flow

```
Client → POST /api/quiz/start → Create attempt →
Return shuffled questions (no correct answers) →
Client displays quiz → User answers →
POST /api/quiz/submit → Edge Function (grade-quiz) →
Server-side grading → Return score + cooldown
```

5.3 Database Schema Overview

```
auth.users ──1:1──→ profiles ──1:N──→ enrollments ──N:1──→ courses
     │                    │                                    │
     │                    │                                    │
     ├──1:N──→ quiz_attempts          courses ──1:N──→ lessons ──1:N──→ resources
     │                    │                   │
     │                    │                   └──1:N──→ questions
     ├──1:N──→ course_grades            courses ──N:N──→ course_prerequisites
     │
     ├──1:N──→ semester_gpas
     │
     ├──1:N──→ cumulative_gpas
     │
     ├──1:N──→ capstone_submissions
     │
     ├──1:N──→ suggestions ──1:N──→ suggestion_votes
     │
     ├──1:N──→ subscriptions
     │
     ├──1:N──→ transactions
     │
     ├──1:N──→ standalone_purchases ──N:1──→ standalone_courses
     │                                                  │
     │                                                  └──1:N──→ standalone_lessons
     │
     └──1:N──→ standalone_lesson_progress
```

Key Tables Summary

TABLE	PURPOSE	ROW COUNT (EST.)
profiles	User profiles with roles	100–1,000
courses	University courses	31+
lessons	Course lessons	200+
questions	Quiz/exam question bank	1,000+
quiz_attempts	Student quiz attempts	1,000+
course_grades	Final grades per course	500+
standalone_courses	Pay-per-course offerings	5–20
standalone_purchases	Course purchase records	100+
subscriptions	Premium subscriptions	50+
transactions	Payment records	200+
audit_log	Admin action log	500+

5.4 Deployment Architecture

```
┌──────────────────────────────────────────┐
│              Vercel                      │
│                                          │
│  Production: aorthar.com                 │
│  Staging: staging.aorthar.com            │
│  Courses: courses.aorthar.com            │
│                                          │
│  Environment Variables:                  │
│  - NEXT_PUBLIC_SUPABASE_URL              │
│  - NEXT_PUBLIC_SUPABASE_ANON_KEY         │
│  - SUPABASE_SERVICE_ROLE_KEY             │
│  - PAYSTACK_SECRET_KEY                   │
│  - PAYSTACK_PUBLIC_KEY                   │
│  - NEXT_PUBLIC_PAYSTACK_PUBLIC_KEY       │
│  - PAYSTACK_WEBHOOK_SECRET               │
│  - GEMINI_API_KEY                        │
│  - RESEND_API_KEY                        │
│  - NEXT_PUBLIC_APP_ENV                   │
└──────────────────────────────────────────┘
              │
      ┌──────▼──────┐
      │  Supabase   │
      │  (Production)│
      │             │
      │  Project URL│
      │  Database   │
      │  Auth       │
      │  Edge Functions│
      │  Storage    │
      └─────────────┘
```

5.5 Security Model

Authentication

- Supabase Auth (email/password)
- Email verification required

- Session cookies with `.aorthar.com` domain for cross-subdomain access
- httpOnly, secure, sameSite=lax cookies

Authorization

- Role-based: student, contributor, admin
- Admin levels: super_admin, content_admin, finance_admin
- Row Level Security (RLS) on all tables
- `is_admin()` and `is_premium()` helper functions
- Admin route guard (disabled in development)

Data Protection

- Service role key only used server-side
- RLS prevents cross-user data access
- Audit log is append-only
- Payment data stored with Paystack references (no card data)

5.6 Performance Considerations

AREA	CURRENT STATE	TARGET
Page load (SSR)	1–3s	< 1s
API response	200–500ms	< 200ms
Quiz grading	1–3s (Edge Function)	< 1s
Email delivery	1–30s (Resend)	< 10s
Video load	Depends on YouTube/Drive	N/A (external)
Admin table load	Full query (limit 100)	Paginated

6. Gap Analysis

6.1 Critical Gaps

ID	GAP	IMPACT	MODULE	RECOMMENDATION
G-01	`notifications` table exists but has no UI	Medium	Notifications	Build notification bell in Navbar + notification list page
G-02	No email notification when suggestion is approved/rejected	Medium	Suggestions	Send email to proposer on status change
G-03	No email notification when capstone is reviewed	Medium	Capstone	Send email to student on review
G-04	`?module=courses` query param on admin pages has no effect	Low	Admin	Either implement filtering or remove the param from URLs
G-05	No pagination on admin users/transactions tables (limit 100)	Medium	Admin	Add pagination or infinite scroll for large datasets
G-06	No export functionality for transactions/users	Medium	Admin	Add CSV export for financial reporting
G-07	No real-time quiz submission confirmation	Low	University	Add loading state and explicit success/error feedback

6.2 Important Gaps

ID	GAP	IMPACT	MODULE	RECOMMENDATION
G-09	No bulk email capability for admins	Medium	Admin	Add ability to send announcements to all students
G-10	No analytics dashboard (page views, engagement, drop-off)	Medium	Admin	Add basic analytics or integrate with PostHog/Plausible
G-11	No referral/affiliate system for standalone courses	Low	Standalone	Add referral codes for organic growth
G-12	No certificate of completion for standalone courses	Medium	Standalone	Generate PDF certificate when all lessons completed
G-13	No search functionality for courses (university or standalone)	Medium	Both	Add course search with filters
G-14	No mobile app or PWA support	Low	Platform	Consider PWA for offline lesson viewing
G-15	No discussion forum or community feature	Medium	University	Add course-level discussion threads
G-16	No progress email reminders for inactive students	Low	Engagement	Weekly email nudges for students who haven't studied
G-17	No A/B testing framework for pricing page	Low	Monetization	Test pricing card layouts and copy
G-18	Standalone course purchase recording has 3 code paths (redirect, webhook, learn page)	Medium	Standalone	Consolidate to single source of truth

6.3 Nice-to-Have Gaps

ID	GAP	IMPACT	MODULE	RECOMMENDATION
G-19	No dark/light theme toggle on standalone courses platform	Low	Standalone	Add theme toggle matching university
G-20	No course reviews/ratings from students	Low	Standalone	Add star ratings and written reviews
G-21	No wishlist or save-for-later for courses	Low	Standalone	Allow users to bookmark courses
G-22	No social sharing for course completion	Low	Engagement	Share certificate on LinkedIn/Twitter
G-23	No instructor dashboard for standalone courses	Low	Admin	Allow instructors to manage their own courses
G-24	No coupon/discount code system	Medium	Payments	Add promo codes for marketing campaigns
G-25	No multi-currency support	Low	Payments	Currently NGN only
G-26	No student leaderboard or gamification	Low	Engagement	Add streaks, badges, leaderboards

6.4 Broken / Incomplete Flows

ID	FLOW	ISSUE	SEVERITY
B-01	Admin users page → standalone buyers	Was not joining standalone_purchases table	**Fixed** (2026-04-03)
B-02	Admin payments page → standalone purchases	Was querying `created_at` instead of `purchased_at`	**Fixed** (2026-04-03)
B-03	Admin users/payments → buyers without profiles	Inner join on profiles excluded buyers without profile rows	**Fixed** (2026-04-03)
B-04	University pages (`/university/*`)	Removed; all university links point to `university.aorthar.com`	**Resolved**
B-05	`?module=courses` param	Present in URLs but not implemented in code	**Low priority**

6.5 Mocked / Placeholder Logic

ID	AREA	STATUS	NOTES
M-01	Demo mode snapshots	Mocked data	`adminSnapshot.ts` and `studentSnapshot.ts` provide fallback when tables are empty
M-02	AI question generation	Real (Gemini)	Works but quality depends on prompt engineering
M-03	Lesson summaries/related/deep-dive	Real (Gemini)	AI-generated, may produce inaccurate links
M-04	YouTube URL validation	Real (oEmbed)	May fail for unlisted/private videos
M-05	Transcript HTML generation	Real	Server-side HTML generation
M-06	Internship product	**Not Built**	Database schema exists, only landing page implemented

6.6 Backend Gaps

ID	GAP	IMPACT	RECOMMENDATION
BG-01	No rate limiting on API routes	Medium	Add rate limiting for auth, quiz, and payment endpoints
BG-02	No API versioning	Low	Add `/api/v1/` prefix for future compatibility
BG-03	No webhook retry logic	Medium	If webhook processing fails, no automatic retry
BG-04	No background job queue	Low	Email sending is fire-and-forget; consider a queue for reliability
BG-05	No database backups automation	High	Ensure Supabase automated backups are enabled
BG-06	No migration rollback capability	Medium	Supabase migrations are forward-only

7. Technical Debt Analysis

7.1 Structural Issues

ID	ISSUE	LOCATION	SEVERITY	FIX
TD-01	**Triple purchase recording paths** — `standalone_purchases` can be recorded via: (1) `/api/standalone/verify-payment`, (2) `/api/webhooks/paystack`, (3) `/courses-app/learn/[slug]/page.tsx`. Creates maintenance burden and subtle inconsistency risks.	Standalone checkout	High	Consolidate to single recording function; use webhook as primary, redirect as fallback
TD-02	**Inconsistent timestamp columns** — `standalone_purchases` uses `purchased_at`, while `transactions` uses `created_at` and `paid_at`. Other tables use `created_at`.	Database schema	Medium	Standardize on `created_at` for record creation, add domain-specific timestamps as needed
TD-03	**AI library misnamed** — `src/lib/ai/openai.ts` actually calls Gemini API, not OpenAI.	`src/lib/ai/openai.ts`	Low	Rename to `gemini.ts` or `ai.ts`

7.2 Repeated Logic

ID	ISSUE	LOCATION	SEVERITY	FIX
TD-06	**Email sending pattern repeated** — Welcome, password reset, and purchase confirmation emails each have their own API route with similar boilerplate.	`/api/auth/send-welcome`, `/api/auth/forgot-password`, webhook	Low	Create a shared email dispatch utility
TD-07	**Profile lookup pattern repeated** — `profiles` table queried with `user_id` in dozens of places.	Throughout codebase	Low	Create a `getProfileByUserId(userId)` helper
TD-08	**Domain detection logic repeated** — `window.location.hostname.includes('courses.')` checked in multiple components.	Register, login, checkout pages	Low	Create a `isCoursesDomain()` utility

7.3 Hardcoded Values

ID	VALUE	LOCATION	SEVERITY	FIX
TD-09	Instructor name `'Adedamola Adewale'` hardcoded as default in `standalone_courses` table	Database default	Low	Move to environment variable or profiles table
TD-10	Dashboard URL `'https://courses.aorthar.com'` hardcoded in email templates	Email templates	Medium	Use environment variable for base URL
TD-11	Preview duration `60` seconds hardcoded in CourseWatch component	`CourseWatch.tsx`	Low	Make configurable per course
TD-12	Grade scale hardcoded in `src/types/index.ts`	`GRADE_SCALE` constant	Low	Move to database or config file for flexibility
TD-13	Department list hardcoded in `src/lib/academics/departments.ts`	Departments file	Medium	Consider database-driven departments for flexibility

7.4 State Management Issues

ID	ISSUE	LOCATION	SEVERITY	FIX
TD-14	**No global state** — All state is local component state. As the app grows, this will lead to prop drilling and redundant API calls.	Throughout	Medium	Introduce a lightweight state solution (Zustand or React Context) for user session, theme, and demo mode
TD-15	**sessionStorage for checkout flow** — `sessionStorage.setItem(checkoutKey, '1')` used to prevent auto-restart of checkout. Fragile across tabs.	`checkout/[slug]/page.tsx`	Low	Consider URL-based state or cookie

7.5 Scalability Risks

ID	RISK	IMPACT	SEVERITY	MITIGATION
TD-16	**No pagination on admin tables** — Users and transactions loaded with `.limit(100)`. Will break at scale.	Admin UX	Medium	Add cursor-based pagination
TD-17	**No database indexing on hot query paths** — Some queries may become slow as data grows.	Performance	Medium	Verify indexes exist; add composite indexes where needed
TD-18	**Fire-and-forget email sending** — Emails sent without retry or queue. If Resend is down, emails are lost.	Reliability	Medium	Add retry logic or use a queue (Upstash/SQS)
TD-19	**No CDN caching for static assets** — Course thumbnails, logos served directly.	Performance	Low	Configure Vercel Image Optimization or CDN caching
TD-20	**Single Edge Function for grading** — `grade-quiz` Edge Function handles all quiz grading. May become a bottleneck.	Performance	Low	Monitor execution time; consider batching

7.6 Security Risks

ID	RISK	IMPACT	SEVERITY	MITIGATION
TD-21	**Admin guard disabled in development** — `NEXT_PUBLIC_APP_ENV=development` bypasses admin check. If accidentally deployed to production with this value, any user can access admin.	Access control	High	Use server-side env check, not client-side `NEXT_PUBLIC_` variable
TD-22	**No rate limiting** — Auth endpoints, quiz endpoints, and API routes have no rate limiting. Brute force and abuse possible.	Security	Medium	Add rate limiting (Upstash Ratelimit or Vercel Edge Middleware)
TD-23	**Paystack metadata not validated server-side** — `metadata.type`, `metadata.course_id`, `metadata.user_id` trusted from Paystack without additional validation.	Payment integrity	Medium	Add server-side validation of metadata against expected values
TD-24	**No CSRF protection on forms** — Client-side forms don't have CSRF tokens.	Security	Low	Supabase handles this via session cookies, but verify
TD-25	**Audit log IP address may be inaccurate behind proxy** — `ip_address` field in audit_log may show proxy IP instead of real IP.	Audit accuracy	Low	Use `x-forwarded-for` header parsing

7.7 Code Quality

ID	ISSUE	LOCATION	SEVERITY	FIX
TD-26	**No test coverage** — Vitest is configured but no tests exist. `bun run build` is the only correctness gate.	Entire codebase	High	Add unit tests for core logic (GPA, progression, grading) and integration tests for API routes
TD-27	**Type assertions (`as unknown`)** — Used in payments page to cast Supabase join results. Fragile.	`payments/page.tsx`	Medium	Create proper TypeScript types for joined queries
TD-28	**Large page components** — Some page files exceed 200 lines with mixed data fetching and rendering logic.	Multiple pages	Low	Extract data fetching into separate functions
TD-29	**No error boundaries** — React error boundaries not implemented. A crash in one component can take down the whole page.	Throughout	Medium	Add error boundaries at page and layout level
TD-30	**Console.error used for production errors** — Errors logged to console but not sent to monitoring.	Throughout	Medium	Integrate Sentry or similar error tracking

7.8 Priority Fix Order

Priority Fix Order

1. **TD-21** — Admin guard bypass risk (security)
2. **TD-26** — No test coverage (quality)
3. **TD-01** — Triple purchase recording (reliability)
4. **TD-22** — No rate limiting (security)
5. **TD-16** — No pagination on admin tables (scalability)
6. **TD-18** — Fire-and-forget emails (reliability)
7. **TD-04** — Inconsistent timestamp columns (maintainability)
8. **TD-30** — No error monitoring (observability)

8. One-Year Roadmap

8.1 Q1 (Apr–Jun 2026) — Stabilization & Core Fixes

Goals

Fix all critical bugs and gaps. Establish testing foundation. Improve admin experience for monitoring buyers and transactions.

FEATURE	PRIORITY
Admin transaction visibility — ensure all standalone buyers appear in users list and payments page	High
Admin pagination — Add pagination to users, transactions, and suggestions tables	High
CSV export — Export users and transactions from admin panel	High
Email notifications for reviews — Notify students when suggestions/capstones are reviewed	High
Rate limiting — Add rate limiting to auth, quiz, and payment endpoints	High
Error monitoring — Integrate Sentry for production error tracking	High
Test suite — Add unit tests for GPA calculation, progression logic, and grading	High

Technical Priorities: Consolidate standalone purchase recording · Standardize timestamp column naming · Add error boundaries to all page layouts · Fix admin guard to use server-side env check

Success Metrics: 0 critical bugs in production · Test coverage > 40% · Admin page load time < 2s with 1000+ users · Email delivery rate > 99%

8.2 Q2 (Jul–Sep 2026) — Core Feature Expansion

Goals

Enhance the learning experience for both university and standalone students. Build community features. Improve course discovery.

FEATURE	PRIORITY
Course search — Full-text search across university and standalone courses	High
Discussion forums — Course-level discussion threads for university courses	High
Certificate of completion — PDF certificates for standalone courses (all lessons completed)	High
Notification system — Build UI for existing `notifications` table; notification bell in Navbar	High
Course reviews/ratings — Star ratings and written reviews for standalone courses	Medium
Progress email reminders — Weekly nudges for inactive students	Medium
Bulk email for admins — Send announcements to all students or segments	Medium
Coupon/discount codes — Promo code system for marketing campaigns	Medium

Technical Priorities: Introduce lightweight global state (Zustand) · Add React Query or SWR for client-side data fetching · Implement database connection pooling optimization · Add API versioning (`/api/v1/`)

Success Metrics: 20% increase in course completion rate · 15% increase in standalone course conversions · 50+ discussion threads active per month · Notification open rate > 30%

8.3 Q3 (Oct–Dec 2026) — Growth & Monetization

> **Goals**
> Drive revenue growth through new monetization features. Expand standalone course catalog. Build referral and affiliate systems.

FEATURE	PRIORITY
Referral program — Referral codes for standalone courses with discounts	High
Affiliate dashboard — Track affiliate earnings and referrals	High
Instructor onboarding — Allow external instructors to create and manage standalone courses	High
Course bundles — Bundle multiple standalone courses at a discount	Medium
Gift courses — Purchase courses as gifts for other users	Medium
Multi-currency support — Support USD in addition to NGN	Medium
A/B testing framework — Test pricing page layouts and copy	Medium
Analytics dashboard — Page views, engagement metrics, drop-off analysis (PostHog integration)	High

Technical Priorities: Multi-tenant instructor isolation in database · Payment gateway abstraction (support multiple providers) · CDN optimization for course thumbnails and media · Database read replicas for analytics queries

Success Metrics: 30% of standalone revenue from referrals/affiliates · 5+ external instructors onboarded · 20% increase in average revenue per user · Analytics dashboard with < 5min data freshness

8.4 Q4 (Jan–Mar 2027) — Scale & Optimization

Goals

Prepare for 10x user growth. Optimize performance and reliability. Explore new product opportunities.

FEATURE	PRIORITY
PWA support — Offline lesson viewing for standalone courses	High
Mobile app — React Native app for iOS and Android	Medium
Gamification — Streaks, badges, leaderboards for university students	Medium
Social sharing — Share certificates and achievements on LinkedIn/Twitter	Low
Advanced analytics — Cohort analysis, LTV prediction, churn modeling	Medium
API for partners — Public API for university partnerships	Medium
White-label option — Allow institutions to white-label the platform	Low
AI tutor — Gemini-powered Q&A assistant for course content	Medium

Technical Priorities: Microservices architecture for quiz grading and payment processing · Event-driven architecture with message queue (Upstash/SQS) · Database sharding strategy for multi-tenant support · CI/CD pipeline with automated testing and staging deployments · Disaster recovery plan with cross-region backups

Success Metrics: 10,000+ active students · 99.9% uptime · < 1s page load time globally · 50+ courses in standalone catalog · 5+ institutional partnerships

8.5 Out of Scope (Explicitly)

ITEM	REASON
Live video streaming	Requires significant infrastructure; YouTube/Drive is sufficient for now
Real-time collaboration	Not aligned with self-paced learning model
Blockchain/certificates	Over-engineering; PDF certificates are sufficient
Multi-language support	English-only market for now
Corporate/B2B sales	Focus on individual learners first
Video hosting infrastructure	YouTube and Google Drive handle this adequately

9. Product Recommendations

9.1 High-Impact, Low-Effort (Do First)

#	RECOMMENDATION	IMPACT	EFFORT
1	**Admin CSV Export** — Add export button to users and payments pages for finance reconciliation	High	2–4 hours
2	**Email Notifications for Reviews** — Notify students on suggestion/capstone review status change	High	4–6 hours
3	**Admin Pagination** — Add pagination to all admin tables (users, transactions, suggestions)	High	4–8 hours
4	**Consolidate Purchase Recording** — Single source of truth for standalone purchases (webhook primary, redirect fallback)	High	8–12 hours
5	**Rate Limiting** — Protect auth, quiz, and payment endpoints from brute force and abuse	High	4–8 hours

9.2 High-Impact, Medium-Effort (Do Next)

#	RECOMMENDATION	IMPACT	EFFORT
6	**Course Search** — Full-text search across university and standalone courses with filters (department, level, price, status)	High	1–2 weeks
7	**Certificate of Completion** — PDF certificates for standalone courses when all lessons completed	High	1 week
8	**Notification System UI** — Bell icon in Navbar + notification list page (uses existing `notifications` table)	Medium	1–2 weeks
9	**Coupon/Discount Codes** — Promo codes for marketing campaigns, flash sales, referral programs	High	1–2 weeks
10	**Analytics Dashboard** — PostHog integration for page views, event tracking, funnel analysis, cohort retention	High	1 week

9.3 Medium-Impact, Medium-Effort (Plan for Q2)

#	RECOMMENDATION	IMPACT	EFFORT
11	**Discussion Forums** — Course-level discussion threads for university courses	Medium	2–3 weeks
12	**Progress Email Reminders** — Weekly automated emails to students inactive for 7+ days	Medium	1 week
13	**Bulk Email for Admins** — Send announcements to all students or filtered segments	Medium	1–2 weeks
14	**Course Reviews/Ratings** — Star ratings and written reviews for standalone courses	Medium	1–2 weeks
15	**Referral Program** — Referral codes that give both referrer and referee a discount	Medium	2 weeks

9.4 Strategic Recommendations

#	RECOMMENDATION	WHY
16	**Instructor Marketplace** — Allow external instructors to create and sell their own courses (20–30% revenue share)	Transforms from single-instructor to marketplace, dramatically increasing catalog size and revenue potential
17	**Corporate/B2B Licensing** — Bulk licenses for company teams with HR admin dashboard	Higher ARPU than individual sales; recurring B2B revenue
18	**Mobile App (PWA First)** — Progressive Web App with offline lesson viewing before native app	Lower development cost, instant distribution, works on all devices
19	**AI Tutor** — Gemini-powered Q&A assistant for course content	Differentiates from competitors, increases learning outcomes
20	**White-Label for Universities** — B2B SaaS model allowing institutions to white-label the platform	Recurring revenue from institutional partnerships

9.5 Quick Wins (Under 2 Hours Each)

#	CHANGE	IMPACT
QW-01	Rename `src/lib/ai/openai.ts` to `gemini.ts`	Reduces confusion for developers
QW-02	Add `rel="noopener noreferrer"` to all external links	Security best practice
QW-03	Add Open Graph meta tags to all public pages	Better social sharing
QW-04	Add loading skeletons to admin tables	Perceived performance improvement
QW-05	Add empty state illustrations	Better UX for empty states
QW-06	Standardize date formatting across all pages	Consistency
QW-07	Add `?module=courses` filtering logic or remove param	Eliminates confusion
QW-08	Add transcript PDF generation (currently HTML)	Professional output format

10. Scope Control

10.1 In Scope (Current Product)

Core Platform

- ✅ Email/password authentication with verification
- ✅ Password reset via email
- ✅ Student onboarding (department selection)
- ✅ 4-year university curriculum (100–400 level)
- ✅ 8 departments with semester-based progression
- ✅ Quiz/exam engine with server-side grading
- ✅ GPA calculation (5.0 scale)
- ✅ Capstone submission and review

- ✅ Community suggestions system
- ✅ Premium subscriptions (Paystack)
- ✅ Standalone courses (pay-per-course)
- ✅ YouTube and Google Drive video embedding
- ✅ Admin CMS (courses, users, payments, pricing)
- ✅ Multi-level admin permissions (super, content, finance)
- ✅ Audit logging
- ✅ Account suspension
- ✅ Email delivery (Resend)
- ✅ AI lesson content (Gemini)
- ✅ Demo mode
- ✅ Transcript HTML generation
- ✅ Partnership inquiries
- ✅ Lesson interactions (reactions, comments)

Admin Features

- ✅ Course CRUD (university + standalone)
- ✅ Lesson and resource management
- ✅ Question bank management
- ✅ User management (search, filter, suspend, grant premium)
- ✅ Transaction monitoring (subscriptions + standalone)
- ✅ Suggestion review
- ✅ Capstone review
- ✅ Pricing configuration
- ✅ Audit log viewer
- ✅ Student invite and bulk import
- ✅ Admin access management

10.2 Out of Scope (Explicitly Not Building)

Infrastructure

- ❌ Live video streaming infrastructure (using YouTube/Drive)
- ❌ Video hosting/encoding (using YouTube/Drive)
- ❌ CDN infrastructure (using Vercel)
- ❌ Database infrastructure (using Supabase)

Product Features

- ❌ Real-time collaboration or live classes
- ❌ Blockchain-based certificates
- ❌ Multi-language support (English only)
- ❌ Native mobile apps (PWA considered in Q4)
- ❌ Corporate/B2B sales platform (Q3+ consideration)
- ❌ White-label platform (Q4 consideration)
- ❌ Discussion forums (Q2 consideration)
- ❌ Referral/affiliate system (Q3 consideration)
- ❌ Coupon/discount codes (Q2 consideration)
- ❌ Course reviews/ratings (Q2 consideration)
- ❌ Gamification (Q4 consideration)
- ❌ AI tutor (Q4 consideration)
- ❌ Instructor marketplace (Q3 consideration)
- ❌ Internship product (documented, not yet built)

Technical

- ❌ API versioning (Q2 consideration)
- ❌ Microservices architecture (Q4 consideration)
- ❌ Database sharding (Q4 consideration)
- ❌ Multi-region deployment (Q4 consideration)

10.3 Grey Area (Needs Clarification)

ITEM	STATUS	NOTES
`university.aorthar.com` subdomain	Correct	All university links now point to the subdomain, not `/university/*` paths. Same routes as main domain.
`?module=courses` query param	Unused	Present in URLs but not implemented. Implement or remove.
In-app notifications UI	Table exists, no UI	`notifications` table exists but no frontend. Build or remove?
External instructor accounts	Not yet designed	`instructor_name` is hardcoded. Should instructors have their own accounts?
Course sort order	Partial	`sort_order` column added but not fully utilized in UI
Transcript format	HTML, not PDF	Currently generates HTML file. PDF generation would be an enhancement.